

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

Fabiano Carvalho Costa Júnior – Matrícula: 2520144
Guilherme Vilas Boas Barbosa – Matrícula: 2411726

Laboratório 02
INF1010 – Estruturas de Dados

Rio de Janeiro

2026

1 Identificação

Alunos: Fabiano Carvalho Costa Júnior e Guilherme Vilas Boas Barbosa

Matrículas: 2520144 / 2411726

Disciplina: INF1010 – Estruturas de Dados

Instituição: PUC-Rio

2 Objetivo

O objetivo deste trabalho é desenvolver um programa em linguagem C para ler uma sequência de 20 chaves inteiras contidas em um arquivo texto (`chaves.txt`) e inseri-las dinamicamente em uma Árvore Binária de Busca (ABB). A estrutura de cada nó armazena a chave, a altura e os ponteiros para as subárvores esquerda e direita.

Após a construção completa da árvore, o programa percorre a estrutura de forma recursiva para calcular e atualizar o campo de altura de cada nó. Por fim, o sistema exibe no console os percursos em pré-ordem e em ordem simétrica, imprimindo em cada elemento a chave e sua altura calculada no formato `chave(altura)`, garantindo a correta liberação de toda a memória alocada ao término da execução.

3 Estrutura do Programa

- **Estrutura de Dados (No):** Contém os campos inteiros `chave` e `altura`, e dois ponteiros autorreferenciais (`esq` e `dir`) para as subárvores filhas.
- **Módulo de Alocação (`criaNo`):** Aloca memória dinamicamente com `malloc`, valida a alocação, inicializa a altura como 0 e os ponteiros como `NULL`.
- **Módulo de Inserção (`insere`):** Insere recursivamente cada chave mantendo as propriedades da ABB (chaves menores à esquerda, maiores à direita).
- **Módulo de Cálculo de Altura (`calculaAltura`):** Navega recursivamente até as folhas e atualiza o campo `altura` de cada nó como $1 + \max(\text{alturaE}, \text{alturaD})$, considerando altura -1 para ponteiros `NULL`.
- **Módulos de Percurso (`preOrdem` e `simetrica`):** Percorrem a árvore recursivamente exibindo as chaves e suas respectivas alturas formatadas como `chave(altura)`.
- **Módulo de Liberação (`liberaArvore`):** Percorre a estrutura em pós-ordem desalocando a memória de cada nó com `free`.

4 Solução e Código

O código abaixo realiza a leitura do arquivo `chaves.txt`, montagem da árvore, cálculo das alturas, exibição e liberação de memória:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct no {
5     int chave;
6     int altura;
7     struct no *esq;
8     struct no *dir;
9 };
10
11 typedef struct no No;
12
13 No *criaNo(int chave) {
14     No *novo = (No *) malloc(sizeof(No));
15     if (novo == NULL) {
16         printf("Erro ao alocar memoria!\n");
17         exit(1);
18     }
19     novo->chave = chave;
20     novo->altura = 0;
21     novo->esq = NULL;
22     novo->dir = NULL;
23     return novo;
24 }
25
26 No *insere(No *raiz, int chave) {
27     if (raiz == NULL) {
28         return criaNo(chave);
29     }
30     if (chave < raiz->chave) {
31         raiz->esq = insere(raiz->esq, chave);
32     }
33     else if (chave > raiz->chave) {
34         raiz->dir = insere(raiz->dir, chave);
35     }
36     return raiz;
37 }
38
39 int calculaAltura(No *raiz) {
40     if (raiz == NULL) {
41         return -1;
42     }
43     int alturaE = calculaAltura(raiz->esq);
44     int alturaD = calculaAltura(raiz->dir);
45     if (alturaE > alturaD) {
46         raiz->altura = alturaE + 1;
47     }
48     else {
49         raiz->altura = alturaD + 1;
50     }
51     return raiz->altura;

```

```

52 }
53
54 void preOrdem(No *raiz) {
55     if (raiz != NULL) {
56         printf("%d(%d)␣", raiz->chave, raiz->altura);
57         preOrdem(raiz->esq);
58         preOrdem(raiz->dir);
59     }
60 }
61
62 void simetrica(No *raiz) {
63     if (raiz != NULL) {
64         simetrica(raiz->esq);
65         printf("%d(%d)␣", raiz->chave, raiz->altura);
66         simetrica(raiz->dir);
67     }
68 }
69
70 void liberaArvore(No *raiz) {
71     if (raiz != NULL) {
72         liberaArvore(raiz->esq);
73         liberaArvore(raiz->dir);
74         free(raiz);
75     }
76 }
77
78 int main(void) {
79     FILE *arquivo;
80     No *raiz = NULL;
81     int chave;
82
83     arquivo = fopen("chaves.txt", "r");
84     if (arquivo == NULL) {
85         printf("Erro␣ao␣abrir␣o␣arquivo␣chaves.txt\n");
86         return 1;
87     }
88
89     while (fscanf(arquivo, "%d", &chave) == 1) {
90         raiz = insere(raiz, chave);
91     }
92     fclose(arquivo);
93
94     calculaAltura(raiz);
95
96     printf("Pre-ordem:␣");
97     preOrdem(raiz);
98     printf("\n");
99
100    printf("Simetrica:␣");
101    simetrica(raiz);
102    printf("\n");

```

```

103
104     liberaArvore(raiz);
105     return 0;
106 }

```

4.1 Resultados e Saída do Programa

Utilizando o arquivo `chaves.txt` preenchido com as 20 chaves fornecidas:

20 30 70 50 100 60 80 90 10 40 5 95 25 85 35 75 65 55 45 41

A execução do programa no terminal gera rigorosamente a seguinte saída formatada:

```

Pre-ordem: 20(8) 10(1) 5(0) 30(7) 25(0) 70(6) 50(4) 40(2) 35(0)
           45(1) 41(0) 60(1) 55(0) 65(0) 100(5) 80(4) 75(0) 90(3) 85(0)
           95(2)
Simetrica: 5(0) 10(1) 20(8) 25(0) 30(7) 35(0) 40(2) 41(0) 45(1)
           50(4) 55(0) 60(1) 65(0) 70(6) 75(0) 80(4) 85(0) 90(3) 95(2)
           100(5)

```

5 Observações e Conclusões

- **Análise do Resultado:** A impressão dos dados no percurso em Ordem Simétrica é exibida de forma estritamente crescente, o que confirma visualmente que a árvore binária de busca foi construída corretamente.
- **Cálculo de Altura:** A adoção de -1 como caso base para nós nulos permitiu que os nós folha tivessem altura 0, propagando a maior altura das subárvores até a raiz (chave 20, com altura 8).
- **Instruções de Compilação:** O código foi compilado em ambiente GCC através do comando:

```

gcc -Wall -std=c99 main.c -o arvore
./arvore

```

Certifique-se de manter o arquivo `chaves.txt` na mesma pasta do executável.

- **Resultados dos Testes:** O programa executou perfeitamente todos os requisitos solicitados, realizando a leitura do arquivo, o cálculo pós-inserção e a limpeza de memória via desalocação dinâmica sem vazamentos.